Stack

Data Structures and Algorithms
Queue

Mentor: **Adarsh Gupta**
(Senior Software Engineer)

Array, Stack

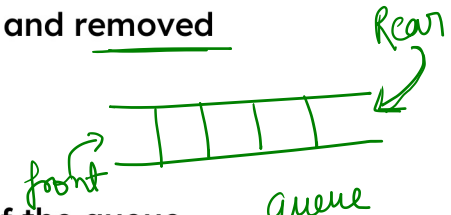
linear

Non - linear

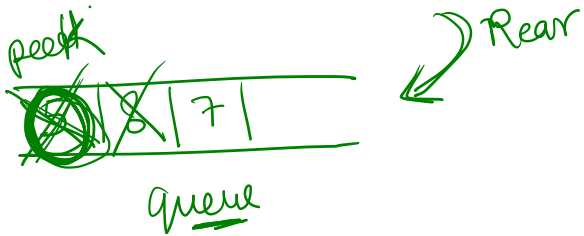
Queue:

Queue is a linear data structure that follows FIFO (First In First Out) Principle, so the first element inserted is the first to be popped out.

- Elements are added to the "rear" (or "tail") of the queue and removed from the "front" (or "head").
- offer(): Adds an element to the rear of the queue.
- poll(): Removes and returns the element from the front of the queue.
- peek() : Returns the element at the front of the queue without removing it.
- isEmpty(): Checks if the queue is empty.
- size() :



true / false



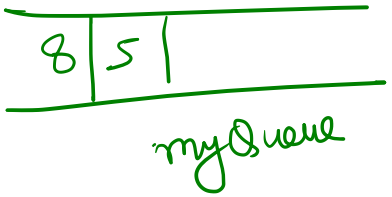
queue.size() →

Integer , Character

Create a Queue using LinkedList

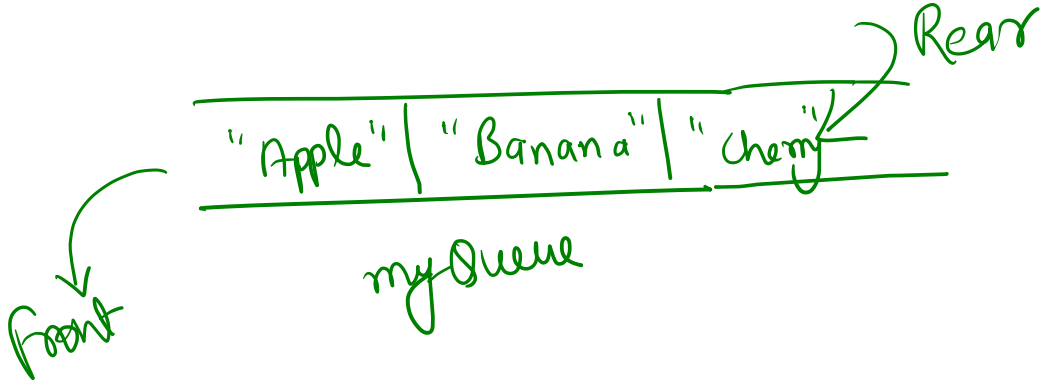
```
Queue<String> myQueue = new LinkedList<>();
```

```
// Adding elements in queue
myQueue.offer("Apple");
myQueue.offer("Banana");
myQueue.offer("Cherry");
```




```
// Peek at the front element
System.out.println("Front element (peek): " + myQueue.peek()); // Apple
```

top / peek



Create a Queue using LinkedList

```
// Dequeue an element
String removedElement = myQueue.poll();
System.out.println("Removed element (poll): " +  removedElement);

// Check if the queue is empty
System.out.println("Is queue empty? " + myQueue.isEmpty());

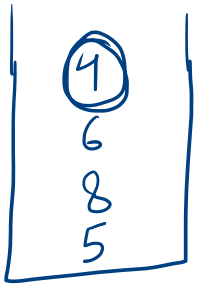
// Get the size of the queue
System.out.println("Queue size: " + myQueue.size());
```



queue → poll(), offer()

Implement Queue using Stacks [Easy]

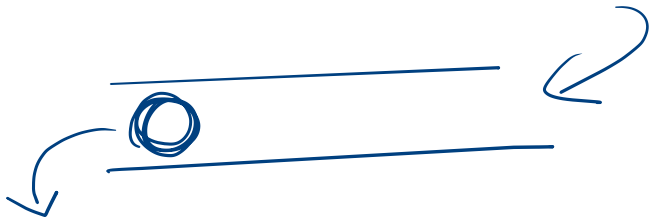
Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).



Stack
→ push()
→ pop()

Implement the MyQueue class:

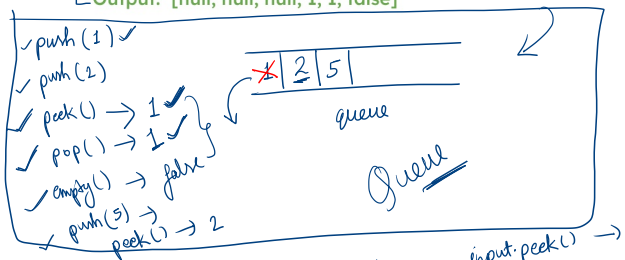
- void push(int x) Pushes element x to the back of the queue.
- int pop() Removes the element from the front of the queue and returns it.
- int peek() Returns the element at the front of the queue.
- boolean empty() Returns true if the queue is empty, false otherwise.



Implement Queue using Stacks

Input: ["MyQueue", "push", "push", "peek", "pop", "empty"]
[[], [1], [2], [], [], []]

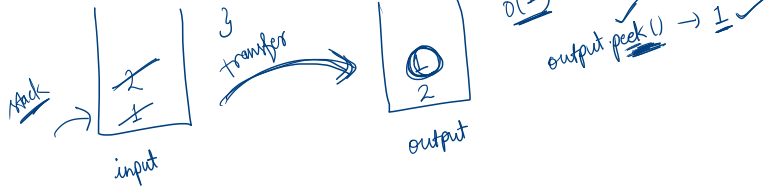
Output: [null, null, null, 1, 1, false]



peekElement = 1

while(!input.isEmpty()) {
output.push(input.pop())
}

input.peek() → 2 ✗





for empty → input.isEmpty() && output.isEmpty()

* If output is empty then transfer from input to output stack.

input → output

O(n)
O(1)
⇒ Amortized O(1)